FEAT-ZD-48600 · DS-04 Identify the components/layers to be changed · Components, layers, sources or pipelines to change

Data Science agent · item task.components · source hobasa-data-science-agent/outputs/FEAT-ZD-48600/task.components.md · approved by automated checks and independent judge

task.components: FEAT-ZD-48600 (Data Science lane, ap_forecast)

DS-04 Identify the components/layers to be changed (attempt 1). Workbook action: read the existing code, understand it, and list the components and layers to create or update with the minimum lines of code. Also confirm secrets access in all environments.

Result: the lane changes one repository, ap_forecast. It needs one new module (`risk.py`), additive edits to two existing modules (`explain.py`, `__init__.py`), one new test file (`tests/test_risk.py`) and new tests in `tests/test_forecast.py`. `predict.py` and `models.json` stay unchanged. No database or Databricks change is needed, and the change adds no secret (section 5).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DS-04 Identify the components/layers to be changed (attempt 1)	record:brief
Lane / repository	Data Science / ap_forecast	feat.tasks (section 1); task.repo_access (section 1)
Tasks in scope	TSK-03 (STORY-03), TSK-04 (STORY-04)	feat.tasks (sections 1, 3)
Inputs	feat.tasks (b0fd4d2), feat.stories (0590ea4), feat.prd (908b58d), feat.trd (3637e51), feat.design (722a98f), task.repo_access (c7b4be8)	record:brief
Code read	`src/ap_forecast/__init__.py`, `explain.py`, `predict.py`, `models.json`, `tests/test_forecast.py`	tool:repo
Working branch	`feature/story`, created from `dev` at `d67e866`, clean	task.repo_access (section 2)

1. What the code does today

Layer	Module	Today	Source
Package API	`src/ap_forecast/__init__.py`	Imports `predict_delay_days` and `build_explanation_request`; `__version__ = "0.9.0"`; `__all__` lists those two names	tool:repo
Prediction (rule / model)	`src/ap_forecast/predict.py`	`predict_delay_days(history)`: rounded mean of past delays; 0 for an empty history; negative delays count as 0	tool:repo

Layer	Module	Today	Source
LLM request	`src/ap_forecast/explain.py`	`build_explanation_request(vendor, predicted_days, config=None)` builds a one-sentence prompt and returns `provider`, `model`, `max_tokens` (from `config or load_config()`) and `messages`	tool:repo
LLM configuration	`src/ap_forecast/models.json`	`provider` `acme-llm`, `model` `acme-text-2`, `max_tokens` 300	tool:repo
Tests	`tests/test_forecast.py`	`test_predict`, `test_request_uses_config`, `test_request_with_explicit_config`	tool:repo
CI	`/usr/bin/python3 ci.py`	Compile, unit tests (unittest), line coverage >= 90%, static checks in place of SonarQube	task.repo_access (section 3)

The package has no network call or pipeline layer of its own. The only external link is the LLM request dict, which `explain.py` builds and returns without sending it (tool:repo). It has no database (journey:task.db_changes, section 2).

2. Components and layers to create or update

#	Layer	Component	Action	Task / story	What changes (minimum)	Source
CMP-1	Prediction (rule)	`src/ap_forecast/risk.py`: `HIGH_ABOVE_DAYS = 30`, `MEDIUM_ABOVE_DAYS = 10`, `vendor_risk(history: Iterable[int]) -> str`	**Create**	TSK-03 / STORY-03	New module. Imports only `predict_delay_days` (not `explain`). Calls it once and compares with strict >	feat.design (HC-4, B2.1, B2.2, DD-9); feat.tasks (TSK-03)
CMP-2	Package API	`src/ap_forecast/__init__.py`	**Update**	TSK-03 / STORY-03	Add `from .risk import vendor_risk`, add `"vendor_risk"` to `__all__`, and take the one minor version step from `0.9.0` (TSK-03 owns it for the package)	feat.design (B2.1, DD-11); feat.tasks (section 2, TSK-03); feat.stories (TK-03.1, TK-03.4)
CMP-3	LLM request	`src/ap_forecast/explain.py`: `build_explanation_request(vendor, predicted_days, config=None, risk=None)`	**Update (additive)**	TSK-04 / STORY-04	Add `risk=None` as the last parameter. Append `f" Risk level: {risk}."` to the prompt only when `risk is not None`. The returned dict construction stays the same	feat.design (HC-5, B2.2); feat.trd (IF-4.1 to IF-4.5); feat.tasks (TSK-04)

#	Layer	Component	Action	Task / story	What changes (minimum)	Source
CMP-4	Tests	`tests/test_risk.py`	**Create**	TSK-03 / STORY-03	`test_vendor_risk_levels`, `test_vendor_risk_boundaries`, `test_vendor_risk_empty_and_early`, `test_vendor_risk_no_llm`, `test_vendor_risk_exported`	feat.tasks (section 2 test files, TSK-03); feat.design (B2.5)
CMP-5	Tests	`tests/test_forecast.py`	**Update (append)**	TSK-04 / STORY-04	Add `test_request_with_risk` and `test_request_without_risk_unchanged`. Keep the existing tests exactly as they are	feat.tasks (TSK-04); feat.design (B2.5); feat.stories (TK-04.2)

Left unchanged

Component	Why	Source
`src/ap_forecast/predict.py`	`vendor_risk` reuses `predict_delay_days` as it is	feat.design (B2.1); feat.tasks (TSK-03 "Must not change")
`src/ap_forecast/models.json`	Provider, model and `max_tokens` are unchanged; `vendor_risk` must not read it	feat.trd (IF-3.2, IF-4.4); feat.tasks (TSK-04 "Must not change")
`load_config` and the `config or load_config()` source in `explain.py`	AC-04.4: provider, model and `max_tokens` still come from `config` or `models.json`	feat.stories (AC-04.4)
Existing signatures	"Keep every existing signature; add new functions."	feat.design (Architecture rule row, NF-4)
Other repositories (ap_pipeline, ap_aging, ap_portal)	Other lanes' tasks. No lane imports another lane's package	feat.design (Part C "Lane dependencies")

3. Minimum-code implementation sketch

The sketches follow the signatures and rules in feat.design B2.2. The coding step (DS-05) writes the final code.

CMP-1, new `risk.py`**.** A single comparison chain with no validation of its own. `predict_delay_days` already applies `int(d)` (feat.tasks TSK-03 notes).

```
"""Rule-based vendor payment-risk level from the predicted delay. No LLM, no models.json read."""
from typing import Iterable

from .predict import predict_delay_days

HIGH_ABOVE_DAYS = 30
MEDIUM_ABOVE_DAYS = 10


def vendor_risk(history: Iterable[int]) -> str:
    """'high' if the predicted delay is above HIGH_ABOVE_DAYS, 'medium' if above MEDIUM_ABOVE_DAYS, else 'low'."""
    days = predict_delay_days(history)
    if days > HIGH_ABOVE_DAYS:
        return "high"
    return "medium" if days > MEDIUM_ABOVE_DAYS else "low"
```

CMP-2, `__init__.py`**.** One import line, one name added to `__all__`, and the `__version__` value changed. The module docstring may mention the risk level.

CMP-3, `explain.py`**.** Add the parameter and one guarded append. Everything else in the function stays the same.

```
def build_explanation_request(vendor: str, predicted_days: int, config=None, risk=None) -> dict:
    cfg = config or load_config()
    prompt = f"Explain in one sentence why vendor {vendor} is predicted to be paid {predicted_days} days late."
    if risk is not None:
        prompt += f" Risk level: {risk}."
    return {...}  # unchanged
```

Why this is the minimum

Choice	Effect on code size	Source
Reuse `predict_delay_days` instead of a second averaging rule	No change to `predict.py`	feat.design (B2.1); feat.trd (IF-3.1)
No input validation in `vendor_risk`; call `predict_delay_days` exactly once, so a generator also works	No extra branches to test	feat.tasks (TSK-03 notes)
`risk` appended without validation (any non-`None` value)	One `if` in `explain.py`	feat.tasks (TSK-04 notes); feat.trd (IF-4.5)
`risk` as the last parameter with default `None`	No caller or existing test changes	feat.trd (IF-4.1); feat.stories (AC-04.3)
Export only `vendor_risk` in `__all__`; the threshold constants stay module-level in `risk.py` (agent judgement)	No extra exports to maintain	feat.design (B2.1); feat.stories (TK-03.1, AC-03.8)

4. Tests and coverage

Test	Asserts	AC	Source
`test_vendor_risk_levels`	Histories predicting 31 → `'high'`, 20 → `'medium'`, 5 → `'low'`	AC-03.1, AC-03.3	feat.design (B2.5)
`test_vendor_risk_boundaries`	Single-value histories `[30]` → `'medium'`, `[31]` → `'high'`, `[10]` → `'low'`, `[11]` → `'medium'`	AC-03.2, AC-03.4	feat.tasks (TSK-03); record:memory (L-007)
`test_vendor_risk_empty_and_early`	`[]` → `'low'`; only negative delays → `'low'`	AC-03.5, AC-03.6	feat.design (B2.5)
`test_vendor_risk_no_llm`	Patch `ap_forecast.explain.load_config` to fail; `vendor_risk` still returns	AC-03.7	feat.tasks (TSK-03)
`test_vendor_risk_exported`	`"vendor_risk" in ap_forecast.__all__`	AC-03.8	feat.design (B2.5)
`test_request_with_risk`	Prompt ends with `' Risk level: high.'`	AC-04.1	feat.tasks (TSK-04)
`test_request_without_risk_unchanged`	Same result with no `risk` and with explicit `risk=None`	AC-04.2, AC-04.5	feat.tasks (TSK-04)
Existing three tests	Pass unchanged (positional `config` still works)	AC-04.3	feat.design (B2.5)

Every new line is reached by these tests. Each branch of `vendor_risk` is hit by the levels and boundaries tests, and both sides of the `risk is not None` guard are hit by the two request tests. The 90% coverage bar is therefore expected to hold (feat.tasks section 2 CI bar; task.repo_access section 3). CI has not run in this environment yet, so the first CI run in DS-05/DS-06 confirms it (task.repo_access RAQ-2).

5. Secrets, database and environments

Item	Finding	Source
Secrets added by this change	None. "No secrets, credentials or personal data are added"	feat.trd (S-6); feat.design (SE-4)
Credentials in ap_forecast code or config	None. `models.json` holds only `provider`, `model` and `max_tokens`	tool:repo
Outbound calls added	None. `vendor_risk` makes no LLM or network call. `build_explanation_request` only builds the request dict, with provider and model unchanged	feat.trd (IF-3.2, IF-4.4); tool:repo
Database / Databricks, all environments	None needed; nothing to provision	journey:task.db_changes (section 3); task.repo_access (section 2)
Per-environment secret grants (local/CI, `feature/<story>`, `dev`, `main`)	Not re-checked in this step. The DS-02 record is not among this step's inputs. Nothing above needs a secret in any of them	Open question CQ-DS-3

The tests need no live provider: they check the built prompt text and patch `load_config` (feat.tasks TSK-03, TSK-04). Sending the risk label to `acme-llm` waits on Architect confirmation (TQ-8). That gates release, not coding (feat.tasks TSK-04; feat.design SE-3).

6. Dependencies and order

Item	Effect on this lane	Source
Other lanes	No code dependency. TSK-03 and TSK-04 can be built in parallel with DE, BE and FE	feat.design (Part C); feat.tasks (section 1)
TSK-03 ↔ TSK-04	Independent. Both touch `ap_forecast`, but different files apart from the shared version step, which TSK-03 owns	feat.tasks (section 2)
PROV-ACME-2026-11 (PQ-5)	Also changes `build_explanation_request`. Whichever merges second rebases and re-runs T-7	feat.tasks (TSK-04); feat.trd (R-3)
End-to-end harness (TSK-08)	Uses `vendor_risk` (CMP-1); does not call `build_explanation_request`	feat.tasks (TSK-08); feat.stories (SA-5)

7. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
CQ-DS-1	feat.stories TK-03.2 suggests adding the risk tests to `tests/test_forecast.py`, while feat.tasks (section 2, TA-4) assigns a new `tests/test_risk.py`	Follow feat.tasks, which is later than feat.stories; the split still awaits Developer confirmation (TKQ-2): new `tests/test_risk.py`	Developer
CQ-DS-2	Exact next version number after `0.9.0`	A minor step, value confirmed in DS-05 (feat.tasks section 2: "The next version number is for the lane to confirm")	Developer
CQ-DS-3	Confirm that the DS-02 per-environment secrets record (local/CI, `feature/<story>`, `dev`, `main`) still shows no secret required for ap_forecast	Nothing in TSK-03 or TSK-04 needs a secret (section 5)	Developer / Shared services
RAQ-1 (carried)	Both stories on the single `feature/story` branch, or per-story branches?	Shared `feature/story`, commits prefixed `STORY-03` / `STORY-04`	Developer

Carried, unchanged: PQ-1 (AC-03.5), TQ-5 (AC-03.6), TQ-4 (AC-04.5), TQ-8 and PQ-5 (TSK-04 release) (feat.stories; feat.tasks).

8. Validation check

Check	Result	Source
Every DS task maps to components	TSK-03 → CMP-1, CMP-2, CMP-4; TSK-04 → CMP-3, CMP-5	feat.tasks (TSK-03, TSK-04)
Files stay inside each task's budget	Yes	feat.tasks (TSK-03, TSK-04 "Files in budget")
Existing signatures kept	Yes (`risk=None` is the last parameter)	feat.design (NF-4); feat.trd (IF-4.1)
No new technology	Python 3.9 standard library only	feat.design (DD-1)
Decisions and lessons	L-007 applied (boundary tests). D-2026-044 / L-011 do not apply: no monetary amount in this lane (agent judgement)	record:memory; feat.tasks (TSK-03)
Blocking questions	None	—